

Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 1 & 2: The OS as a Resource Manager. Building an OS.

Author:
st. gr. FAF-241 Mitrofan Alexia
Verified:
asist. univ. Reițman Patricia

Chișinău - 2026

Contents

LABORATORY 1 **2**

Objectives 2

Before the start 2

Part 1 3

Part 2 6

Part 3 8

Part 4 9

LABORATORY 2 **12**

Objectives 12

Before the start 12

Part 1 12

Part 2 12

Part 3 15

Part 4 19

CONCLUSION **22**

CONCLUSION **23**

LABORATORY 1

Objectives

The goal of this laboratory work is to familiarize oneself with one of the most fundamental parts of any OS, such as the four core jobs: managing files, processes, memory, and devices; and also get used to Linux OS and working in a virtual machine environment for those who use Windows or any other OS.

Before the start

For this specific laboratory, WSL has been used, which is a feature of Windows that allows us to run a Linux environment on our Windows machine, without the need for a separate virtual machine or dual booting. Starting from laboratory 2, Linux in a virtual machine environment will be used.

```
whoami          # who am I to the OS
uname -a        # which kernel is managing this machine
uptime          # how long has this OS been running
```

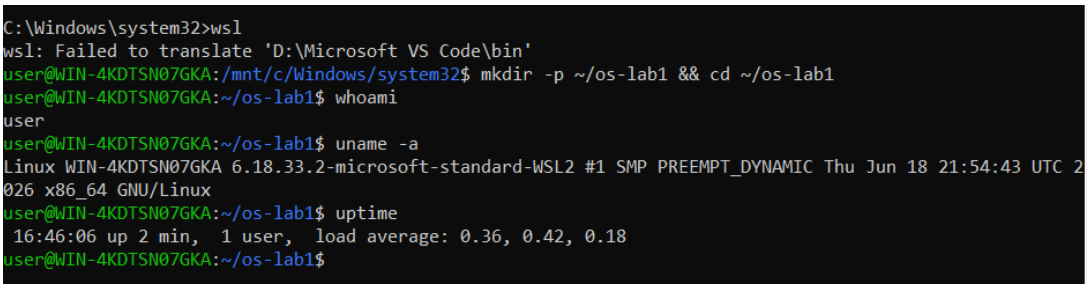


Figure 1: Creating working folder and basic commands

Part 1. Files and directories

```
pwd                     # where am I
ls -la /                # the root of the whole tree
ls -la ~                # my home directory
cd /etc && ls | head
```

```
user@WIN-4KDTSN07GKA:~/os-lab1$ pwd
/home/user/os-lab1
user@WIN-4KDTSN07GKA:~/os-lab1$ ls -la /
total 2844
drwxr-xr-x  20 root root    4096 Sep 28 16:43 .
drwxr-xr-x  20 root root    4096 Sep 28 16:43 ..
drwxr-xr-x   3 root root    4096 Sep 15 18:25 Docker
lrwxrwxrwx   1 root root       7 Apr 20 11:46 bin -> usr/bin
drwxr-xr-x   2 root root    4096 Apr 20 11:46 boot
drwxr-xr-x  15 root root    3820 Sep 28 16:43 dev
drwxr-xr-x  89 root root    4096 Sep 28 16:43 etc
drwxr-xr-x   3 root root    4096 Aug 18 14:36 home
-rwxr-xr-x   1 root root 2837280 Sep 11 08:14 init
lrwxrwxrwx   1 root root       7 Apr 20 11:46 lib -> usr/lib
lrwxrwxrwx   1 root root       9 Apr 20 11:46 lib64 -> usr/lib64
drwx------   2 root root   16384 Aug 18 14:36 lost+found
drwxr-xr-x   2 root root    4096 Apr 20 21:05 media
drwxr-xr-x   5 root root    4096 Aug 18 14:36 mnt
drwxr-xr-x   2 root root    4096 Apr 20 21:05 opt
dr-xr-xr-x 239 root root       0 Sep 28 16:43 proc
drwx------   5 root root    4096 Sep 15 18:25 root
drwxr-xr-x  22 root root     580 Sep 28 16:43 run
lrwxrwxrwx   1 root root       8 Apr 20 11:46 sbin -> usr/sbin
drwxr-xr-x   2 root root    4096 Aug 18 14:36 snap
drwxr-xr-x   2 root root    4096 Apr 20 21:05 srv
dr-xr-xr-x  13 root root       0 Sep 28 16:43 sys
drwxrwxrwt   7 root root     140 Sep 28 16:44 tmp
drwxr-xr-x  12 root root    4096 Apr 20 21:05 usr
drwxr-xr-x  13 root root    4096 Aug 18 14:36 var
user@WIN-4KDTSN07GKA:~/os-lab1$ ls -la ~
total 40
drwxr-x--- 6 user user 4096 Sep 28 16:45 .
drwxr-xr-x 3 root root 4096 Aug 18 14:36 ..
lrwxrwxrwx 1 user user   22 Sep 15 18:25 .aws -> /mnt/c/Users/user/.aws
lrwxrwxrwx 1 user user   24 Sep 15 18:25 .azure -> /mnt/c/Users/user/.azure
-rw------- 1 user user  918 Aug 19 10:57 .bash_history
-rw-r--r-- 1 user user  220 Aug 18 14:36 .bash_logout
-rw-r--r-- 1 user user 3771 Aug 18 14:36 .bashrc
drwxr-x--- 4 user user 4096 Aug 18 14:37 .cache
drwxr-x--- 3 user user 4096 Aug 18 14:37 .config
drwxr-xr-x 4 user user 4096 Sep 15 18:25 .docker
-rw-rw-r-- 1 user user    0 Sep 28 16:43 .motd_shown
-rw-r--r-- 1 user user  807 Aug 18 14:36 .profile
drwxr-xr-x 2 user user 4096 Sep 28 16:45 os-lab1
```

Figure 2: Navigating and inspecting the tree of files (pt. 1)

```
user@WIN-4KDTSN07GKA:~/os-lab1$ cd /etc && ls | head
PackageKit
X11
adduser.conf
alternatives
apparmor
apparmor.d
apport
apt
bash.bashrc
bash_completion
user@WIN-4KDTSN07GKA:/etc$
```

Figure 3: Navigating and inspecting the tree of files (pt. 2)

```
cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

```
user@WIN-4KDTSN07GKA:/etc$ cd ~/os-lab1
user@WIN-4KDTSN07GKA:~/os-lab1$ mkdir demo && cd demo
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ echo "hello operating system" >note.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ echo "hello operating system" > note.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ cp note.txt copy.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ mv copy.txt renamed.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ ls -l
total 8
-rw-r--r-- 1 user user 23 Sep 28 16:50 note.txt
-rw-r--r-- 1 user user 23 Sep 28 16:50 renamed.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ rm renamed.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$
```

Figure 4: Changing the tree: creating, copying, renaming, and removing files

```
ls -l note.txt      # read the rwx / owner / group columns
chmod 600 note.txt  # owner-only read+write
ls -l note.txt
chmod 644 note.txt
```

```
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ ls -l note.txt
-rw-r--r-- 1 user user 23 Sep 28 16:50 note.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ chmod 600 note.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ ls -l note.txt
-rw------- 1 user user 23 Sep 28 16:50 note.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ chmod 644 note.txt
user@WIN-4KDTSN07GKA:~/os-lab1/demo$
```

Figure 5: Looking at ownership and permissions

1. **Who owns the files you just created, and which group?**
   My account "user" owns the file; it can be seen after typing `‘ls -l note.txt‘`. The group "user" owns the file as well.

2. **What do the ten characters at the start of ‘ls -l‘ mean for note.txt after chmod 600?**
   After `‘chmod 600‘` the 10 characters are `‘-rw-------‘`. The 10 characters break down into a 1+3+3+3 format, which represent: file type, owner permissions, group permissions and world permissions.
   The first character is "-", which means that the entry (`‘note.txt‘`) is a regular file (any file that has text/binary data, such as readable files, image files, etc). There can also be "d" for directories, "b" for block files etc. The next 3 characters mean that the owner has read (r) and write (w) permissions set, but no executing permissions set (-), and the remaining dashes mean that group permissions and world permissions are not enabled.
   This is because `‘chmod‘` means "change mode", and we use it to modify file and directory permissions. The "600" part is the numerical value used, the first digit for the owner permissions, the second digit for group permissions, and the third digit for others' permissions. 6 means read (which has a value of 4) and write (which has a value of 2), but not execute (which has a value of 1), and by combining read (4) and write (2) we get 6. The permissions' numerical value for the other 2 groups are 0, which means they have no permissions.

3. **Name two directories directly under / and say, in one line each, what they hold.**
   Two directories under "/" are "home" and "var". On the left of the names, we can read the metadata. The "home" directory is sort of a working space for all users other than root, and there - all those users' home directories are created. The "var" directory is for files that contain variable sizes. For example, there can go databases, log files, etc.

Part 2. Processes

```
ps aux | head
ps aux | wc -l       # how many processes exist right now
top                  # live view; press q to quit
```

```
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.1  24128 15428 ?        Ss   16:43   0:00 /sbin/init
root           2  0.0  0.0   3212  2240 hvc0     Sl+  16:43   0:00 /init
root           6  0.0  0.0   3180  2040 hvc0     Sl+  16:43   0:00 plan9 --control-socket 7 --log-level
4 --server-fd 8 --pipe-fd 10 --log-truncate
root          46  0.0  0.2  50404 16840 ?        S<s  16:43   0:00 /usr/lib/systemd/systemd-journald
systemd+      63  0.0  0.1  22416 14568 ?        Ss   16:43   0:00 /usr/lib/systemd/systemd-resolved
root          86  0.0  0.1  35312 12288 ?        Ss   16:43   0:00 /usr/lib/systemd/systemd-udevd
root         138  0.0  0.0   2888  2016 ?        Ss   16:43   0:00 /bin/sh /usr/lib/systemd/scripts/chro
nyd-starter.sh -n -F 1
root         139  0.0  0.0   4472  3196 ?        Ss   16:43   0:00 /usr/sbin/cron -f -P
message+     140  0.0  0.0   8820  5408 ?        Ss   16:43   0:00 @dbus-daemon --system --address=syste
md: --nofork --nopidfile --systemd-activation --syslog-only
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ ps aux | wc -l
26
user@WIN-4KDTSN07GKA:~/os-lab1/demo$
```

Figure 6: Listing the processes the OS is currently juggling (pt. 1)

```
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ top
top - 16:59:03 up 15 min,  1 user,  load average: 0.00, 0.02, 0.06
Tasks:  24 total,   1 running,  23 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7823.6 total,   7265.5 free,    531.9 used,    176.6 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7291.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   24128  15428  11652 S   0.0   0.2   0:00.92 systemd
      2 root      20   0    3212   2240   2072 S   0.0   0.0   0:00.00 init-systemd(Ub
      6 root      20   0    3180   2040   1940 S   0.0   0.0   0:00.00 init
     46 root      19  -1   50404  16844  15560 S   0.0   0.2   0:00.19 systemd-journal
     63 systemd+  20   0   22416  14568  12076 S   0.0   0.2   0:00.08 systemd-resolve
     86 root      20   0   35312  12288   9248 S   0.0   0.2   0:00.37 systemd-udevd
    138 root      20   0    2888   2016   1884 S   0.0   0.0   0:00.05 chronyd-starter
    139 root      20   0    4472   3196   2944 S   0.0   0.0   0:00.00 cron
    140 message+  20   0    8820   5408   4728 S   0.0   0.1   0:00.08 dbus-daemon
    146 root      20   0   44096  29932  14788 S   0.0   0.4   0:00.24 networkd-dispat
    156 root      20   0   18440   9464   8216 S   0.0   0.1   0:00.12 systemd-logind
    168 syslog    20   0  220548   5824   4652 S   0.0   0.1   0:00.10 rsyslogd
    222 _chrony   20   0   20424   7724   6916 S   0.0   0.1   0:00.04 chronyd
    231 _chrony   20   0   12096   2520   1876 S   0.0   0.0   0:00.00 chronyd
    246 root      20   0    5492   2780   2596 S   0.0   0.0   0:00.01 agetty
    249 root      20   0  123464  32748  18136 S   0.0   0.4   0:00.16 unattended-upgr
    323 root      20   0    3228   1148    980 S   0.0   0.0   0:00.00 SessionLeader
    324 root      20   0    3228   1284   1108 S   0.0   0.0   0:00.12 Relay(325)
    325 user      20   0    6268   5580   3912 S   0.0   0.1   0:00.23 bash
    326 root      20   0    8644   5672   4832 S   0.0   0.1   0:00.01 login
    371 user      20   0   22340  13648  11024 S   0.0   0.2   0:00.16 systemd
    373 user      20   0   22916   4088   2088 S   0.0   0.1   0:00.00 (sd-pam)
    400 user      20   0    6136   5548   3992 S   0.0   0.1   0:00.05 bash
    863 user      20   0    8188   6108   3980 R   0.0   0.1   0:00.00 top
```

Figure 7: Listing the processes the OS is currently juggling (pt. 2)

```
sleep 300 &          # run in the background; note the PID printed
```

```
jobs                  # shell view of background jobs
ps -ef | grep sleep   # find it among all processes
kill <PID>            # ask the OS to terminate it (use the PID above)
```

```
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ sleep 300&
[1] 873
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ jobs
[1]+  Running                 sleep 300 &
user@WIN-4KDTSN07GKA:~/os-lab1/demo$  ps -ef | grep sleep
user         873     325  0 17:00 pts/0    00:00:00 sleep 300
user         893     325  0 17:00 pts/0    00:00:00 grep --color=auto sleep
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ kill 873
[1]+  Terminated              sleep 300
user@WIN-4KDTSN07GKA:~/os-lab1/demo$
```

Figure 8: Starting a background process, finding it, and stopping it

```
sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20   # state, memory, threads
kill $PID
```

```
user@WIN-4KDTSN07GKA:~/os-lab1$ sleep 300 &
[1] 620
user@WIN-4KDTSN07GKA:~/os-lab1$ PID=$!
user@WIN-4KDTSN07GKA:~/os-lab1$ ls /proc/$PID/
arch_status  coredump_filter  gid_map           map_files   ns            personality  smaps         task
attr         cpuset           io                maps        numa_maps     projid_map   smaps_rollup  timens_offsets
auxv         cwd              ksm_merging_pages mem         oom_adj       root         stack         timers
cgroup       environ          ksm_stat          mountinfo   oom_score     sched        stat          timerslack_ns
clear_refs   exe              latency           mounts      oom_score_adj schedstat    statm         uid_map
cmdline      fd               limits            mountstats  pagemap       sessionid    status        wchan
comm         fdinfo           loginuid          net         patch_state   setgroups    syscall
user@WIN-4KDTSN07GKA:~/os-lab1$ cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   620
Ngid:   0
Pid:    620
PPid:   301
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 1000 1001
NStgid: 620
NSpid:  620
NSpgid: 620
NSsid:  301
Kthread:        0
VmPeak:    16112 kB
VmSize:    16112 kB
VmLck:         0 kB
user@WIN-4KDTSN07GKA:~/os-lab1$ kill $PID
[1]+  Terminated              sleep 300
user@WIN-4KDTSN07GKA:~/os-lab1$
```

Figure 9: Looking at what the kernel exposes about a process

1. **What is the PID of process number 1, and what is it (look it up)?**
   PID 1 is ‘`systemd`‘, which is an init system, the first process started by Linux kernel. Its main uses are for device management, login management, network connection management, and event logging.

2. **Roughly how many processes were running on your idle VM?**
   Technically there are a total of 24 processes, but only 1 is running, and the other 23 are sleeping. The ‘`top`‘ process itself is the one running (using the CPU), and the others are waiting for an event to be running. This can be seen in the ”S” column, as every process except top has ”S” (sleeping status), while top has ”R” (running).

3. **From ‘`/proc/<pid>/status`‘, what does the ”State:” line say for your sleeping process?**
   The state is ”S” (sleeping). With ‘`sleep 300 &`‘, we sent the process into background for 300 seconds (5 minutes). After 300 seconds, the process will be finished.

Part 3. Memory

```
free -h                          # total / used / free RAM and swap
cat /proc/meminfo | head -6
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status  # real RAM used by this process
kill $PID
```

```
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       529Mi       7.1Gi       3.5Mi       177Mi       7.1Gi
Swap:          2.0Gi          0B       2.0Gi
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ cat /proc/meminfo | head -6
MemTotal:        8011396 kB
MemFree:         7441020 kB
MemAvailable:    7468824 kB
Buffers:            5648 kB
Cached:           154688 kB
SwapCached:            0 kB
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ sleep 300 & PID=$!
[1] 1030
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ grep VmRSS /proc/$PID/status
VmRSS:      7876 kB
user@WIN-4KDTSN07GKA:~/os-lab1/demo$ kill $PID
[1]+  Terminated                 sleep 300
user@WIN-4KDTSN07GKA:~/os-lab1/demo$
```

Figure 10: View of memory and physical RAM

1. **How much total RAM does the VM have, and how much is free?**
   The VM has a total RAM of 7.64iGiB (= 8011396 kB, or ≈ 8.2GB in decimal units). In free -h, it is rounded to 7.6Gi.
   Strictly free memory (MemFree: 7441020 kB) is about 7.1 GiB (about 7.6 GB). This is memory that nothing is using at the moment. A better measure of what a new program could actually use is available memory (MemAvailable: 7468824 kB, about 7.12 GiB). It is slightly larger because it also counts memory used for buffers and cache that the kernel can reclaim when needed.
   free and /proc/meminfo report the same data, because free reads its numbers from /proc/meminfo. The difference is presentation: free -h converts to readable units and rounds, and it groups several meminfo lines into columns such as buff/cache. Both show that the VM is almost idle, with only about 529 MiB in use.

2. **What is swap, and how much is configured?**
   Swap is one of the space on disk used as an extension of RAM. When there is not enough physical memory, Linux kernel picks pages of memory that have not been used in a while and writes them to the swap area on disk. Here, 2Gi are allocated for the swap.

3. **How much resident memory (VmRSS) does a bare ‘sleep‘ process use, and does that surprise you?**
   The ”sleep” process uses 7.8MB (7876kB). For a process that is supposed to do ”nothing”, the amount is a bit unexpected; however, the function measures the RSS (resident set size), aka it also takes into consideration the running of program itself, the libraries it depends on (such as C), the stack, etc.

Part 4

```
df -h                  # mounted filesystems and free space
lsblk                  # the block devices (disks) themselves
du -sh ~/os-lab1       # size of your work folder
ls -l /dev | head      # devices presented as files
mount | head
```

```
user@WIN-4KDTSN07GKA:~/os-lab1$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            3.9G     0  3.9G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none            3.9G  4.0K  3.9G   1% /mnt/wsl
drivers         477G  410G   67G  87% /usr/lib/wsl/drivers
/dev/sdd       1007G  2.0G  954G   1% /
none            3.9G   32K  3.9G   1% /mnt/wslg
none            3.9G     0  3.9G   0% /usr/lib/wsl/lib
rootfs          3.9G  2.8M  3.9G   1% /init
none            3.9G  496K  3.9G   1% /run
none            3.9G     0  3.9G   0% /run/lock
none            3.9G     0  3.9G   0% /run/shm
none            3.9G  108K  3.9G   1% /mnt/wslg/versions.txt
none            3.9G  108K  3.9G   1% /mnt/wslg/doc
C:\             477G  410G   67G  87% /mnt/c
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           3.9G     0  3.9G   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs           783M   12K  783M   1% /run/user/1000
user@WIN-4KDTSN07GKA:~/os-lab1$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 356.9M  1 disk
sdb    8:16   0 159.4M  1 disk
sdc    8:32   0     2G  0 disk [SWAP]
sdd    8:48   0     1T  0 disk /mnt/wslg/distro
                               /
user@WIN-4KDTSN07GKA:~/os-lab1$ du -sh ~/os-lab1
16K     /home/user/os-lab1
user@WIN-4KDTSN07GKA:~/os-lab1$ ls -l /dev | head
total 0
crw-r--r-- 1 root root     10, 235 Sep 29 23:51 autofs
drwxr-xr-x 2 root root         600 Sep 29 23:51 block
drwxr-xr-x 2 root root         120 Sep 29 23:51 bsg
crw------- 1 root root     10, 234 Sep 29 23:51 btrfs-control
drwxr-xr-x 2 root root        2840 Sep 29 23:51 char
crw------- 1 root root      5,   1 Sep 29 23:51 console
lrwxrwxrwx 1 root root          11 Sep 29 23:51 core -> /proc/kcore
crw------- 1 root root     10, 260 Sep 29 23:51 cpu_dma_latency
crw------- 1 root root     10, 203 Sep 29 23:51 cuse
```

Figure 11: Mounted filesystems, free space, block devices, size of work folder

```
user@WIN-4KDTSN07GKA:~/os-lab1$ mount | head
none on /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2 type overlay (rw,nosuid,nodev,noatime,lowerdir=/modules,upperdir=/lib/modules/6.18.33.2-microsoft-standard-WSL2/r
microsoft-standard-WSL2/rw/work,uuid=on)
none on /mnt/wsl type tmpfs (rw,relatime)
drivers on /usr/lib/wsl/drivers type 9p (ro,nosuid,nodev,noatime,aname=drivers;fmask=222;dmask=222,cache=0x5,access=client,msize=65536,trans=fd,rfd=8,wfd=8)
/dev/sdd on / type ext4 (rw,relatime,discard,errors=remount-ro,data=ordered)
none on /mnt/wslg type tmpfs (rw,relatime)
/dev/sdd on /mnt/wslg/distro type ext4 (ro,relatime,discard,errors=remount-ro,data=ordered)
none on /usr/lib/wsl/lib type overlay (rw,nosuid,nodev,noatime,lowerdir=/gpu_lib_packaged:/gpu_lib_inbox,upperdir=/gpu_lib/rw/upper,workdir=/gpu_lib/rw/work,uuid=on)
rootfs on /init type rootfs (ro,size=3999936k,nr_inodes=999984)
none on /dev type devtmpfs (rw,nosuid,relatime,size=3999936k,nr_inodes=999984,mode=755)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,noatime)
user@WIN-4KDTSN07GKA:~/os-lab1$
```

Figure 12: Mounted filesystems

1. **Which device is your root filesystem "/" mounted on?**
   It is mounted on `‘/dev/sdd (df -h)‘`. The filetype is ext4.

2. **Give one entry from `‘/dev‘| and say what real thing it stands for.`**
   One entry from `‘dev‘` is "console". Console is a terminal device that represents the system console, and it is where the kernel and system messages go.

3. **In one sentence: what does "everything is a file" mean, based on what you saw?**
   "Everything is a file" is an interface design principle, and we can use the same tools (ls, cat, grep) to operate on many different things (processes in `‘proc/<pid>/status‘`, memory info in `‘/proc/meminfo‘`, devices in `‘/dev/console‘`) through the same file interface.

LABORATORY 2

Objectives

The goal of this laboratory is to get familiar with xv6, which is a modern re-implementation of Unix Version 6, in about 9,000 lines of C, running on RISC-V under the QEMU emulator. We built it from source, booted it, used some of the commands and read some of the code, and wrote our own program that runs inside it.

Before the start

For this laboratory, I used Linux inside a virtual machine environment. First of all, I installed git using `sudo apt update \&\& sudo apt install -y git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu`. I also had to install `qemu-system-riscv` to make it work.

```
cd ~/xv6-riscv 2>/dev/null ||
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls                      # kernel/  user/  Makefile  README ...
```

Part 1. Building and booting xv6

```
make qemu
```

Part 2. Using xv6

```
$ ls
$ cat README
$ echo hello xv6
$ ls | grep c         # pipes work here too
$ wc README
$ usertests -q        # optional: the kernel test suite (takes a while)
```

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
console        3 23 0
```

Figure 13: List of all the directories, files and devices

```
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.

ACKNOWLEDGMENTS

xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14,
2000)).  See also https://pdos.csail.mit.edu/6.1810/, which provides
pointers to on-line resources for v6.

The following people have made contributions: Russ Cox (context switching,
locking), Cliff Frey (MP), Xiao Yu (MP), Nickolai Zeldovich, and Austin
Clements.

We are also grateful for the bug reports and patches contributed by
Abhinavpatel00, Takahiro Aoyagi, Marcelo Arroyo, Hirbod Behnam, Silas
Boyd-Wickizer, Anton Burtsev, carlclone, Ian Chen, clivezeng, Dan
Cross, Cody Cutler, Mike CAT, Tej Chajed, Asami Doi,Wenyang Duan,
echtwerner, eyalz800, Nelson Elhage, Saar Ettinger, Alice Ferrazzi,
Nathaniel Filardo, flespark, Peter Froehlich, Yakir Goaron, Shivam
Handa, Matt Harvey, Bryan Henry, jaichenhengjie, Jim Huang, Matúš
Jókay, John Jolly, Alexander Kapshuk, Anders Kaseorg, kehao95,
Wolfgang Keller, Jungwoo Kim, Jonathan Kimmitt, Eddie Kohler, Vadim
Kolontsov, Austin Liew, Bruce Lowekamp, l0stman, Pavan Maddamsetti,
Imbar Marinescu, Yandong Mao, Matan Shabtay, Hitoshi Mitake, Carmi
Merimovich, mes900903, Mark Morrissey, mtasm, Joel Nider, Hayato Ohhashi,
OptimisticSide, papparapa, phosphagos, Harry Porter, Greg Price, Zheng
qhuo, Quancheng, RayAndrew, Jude Rich, segfault, Ayan Shafqat, Eldar
Sehayek, Yongming Shen, Fumiya Shigemitsu, snoire, Taojie, Cam Tenny,
tyfkda, Warren Toomey, Stephen Tu, Alissa Tung, Rafael Ubal, unicornx,
Amane Uehara, Pablo Ventura, Luc Videau, Xi Wang, WaheedHafez, Keiichi
Watanabe, Lucas Wolf, Nicolas Wolovick, wxdao, Grant Wu, x653, Andy
Zhang, Jindong Zhang, Icenowy Zheng, ZhUyU1997, and Zou Chang Wei.

ERROR REPORTS

Please send errors and suggestions to Frans Kaashoek and Robert Morris
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
operating system for MIT's 6.1810, so we are more interested in
```

Figure 14: Readme (pt. 1)

```
BUILDING AND RUNNING XV6

You will need a RISC-V "newlib" tool chain from
https://github.com/riscv/riscv-gnu-toolchain, and qemu compiled for
riscv64-softmmu.  Once they are installed, and in your shell
search path, you can run "make qemu".
$
```

Figure 15: Readme (pt. 2)

```
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
$ usertests -q
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
test openiput: OK
test exitiput: OK
test iput: OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test dirtest: OK
test exectest: OK
test pipe1: OK
test killstatus:
```

Figure 16: Different commands to test xv6

Doing usertests -q is really slow, it doesn't help the fact that it's inside an emulator of an OS inside a VM. I decided to end the process early.

1. **List three programs xv6 ships with.**
   Three programs xv6 ships with are: `grep, init, kill`, which are executable files (programs). We can see they all are type 2 in the image, which means they are regular files (which includes executables).

2. **You just used a pipe (|) inside xv6. Which two OS features must exist for a pipe to work? (Hint: think processes and communication.)**
   In order for the pipe to work, `fork` and `pipe` need to exist. `Fork` is an operation where a process creates a copy of itself, so using it we are able to create separate processes for each side of the |. With `pipe`, we use it to communicate with both ends of the pipe (the read end and write end) through the shared kernel buffer. `Pipe` is called before `fork` so the children inherit the same descriptors of the pipe.

3. **In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?**
   As of now, it doesn't feel very different, but definitely looks like its older ancestor.

Part 3. Reading the source

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
cat user/user.h        # the list of system calls available to programs
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

```
vboxuser@Ubuntu2604:~/xv6-riscv$ sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
```

Figure 17: Viewing first 40 lines of user/ls.c

```
vboxuser@Ubuntu2604:~/xv6-riscv$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
```

Figure 18: Viewing first 30 lines of user/cat.c

```
vboxuser@Ubuntu2604:~/xv6-riscv$ cat user/user.h
#define SBRK_ERROR ((char *)-1)

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);
```

Figure 19: Viewing user file header

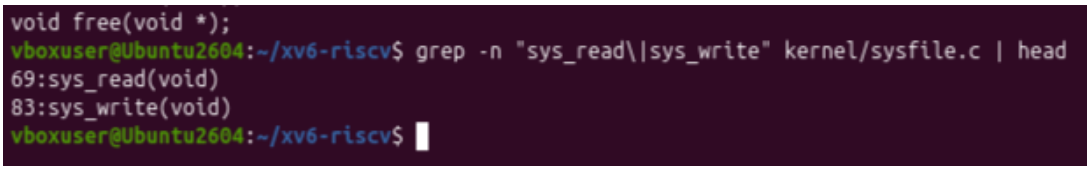


Figure 20: Viewing the starting lines of sys_read and sys_write inside kernel/sysfile.c

1. **Which system calls does user/cat.c use, and what does each one ask the kernel to do?**
   ‘`User/cat.c`‘ uses the following system calls (visible in image 19): open, read, write, exit. Open is used to initialize access to a file; read is used to access data from a file; write is used to write data from a buffer declared by the user to a given device (like file), and exit terminates the calling process.

2. **In which kernel file, and on which line, is sys_read implemented**?
   It is implemented in ‘`kernel/sysfile.c`‘, starting from line 69.

3. **In one sentence: what is the difference between the code in kernel/ and the code in user/?**
   The code in kernel is the code that has the most privileges and is able to control every aspect of the PC, like CPU, all of the memory, I/O, etc. The user code has less privileges, and programs in user mode are ran by the kernel inside private sandbox environments, and can only interface through the kernel.

Part 4. Writing the first xv6 program

I wrote the program with gedit. I had to modify a bit compared to the lab, and use ‘`pause`‘ instead of ‘`sleep`‘ for the system call, because the more recent version of xv6 renamed the ‘`sleep`‘ system call to ‘`pause`‘ to avoid a clash with the kernel’s ‘`sleep`‘.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

Open *sleep.c ~/xv6-riscv/user Save

```c
#include "kernel/types.h"
#include "user/user.h"

int
main (int argc, char *argv[])
{
 if (argc != 2) {
   fprintf(2, "usage: sleep <ticks>\n");
   exit(1);
  }
  pause(atoi(argv[1])); // system call into kernel
  exit(0);
}
```

Figure 21: Writing the program sleep.c

```
UPROGS=\
  $U/_cat\
  $U/_echo\
  $U/_sleep\          # <-- add this line
  ...
```



Figure 22: Adding the sleep.c in the UPROGS list

```
make qemu
$ sleep 10            # should pause, then return to the prompt
$ sleep               # should print your usage message
```

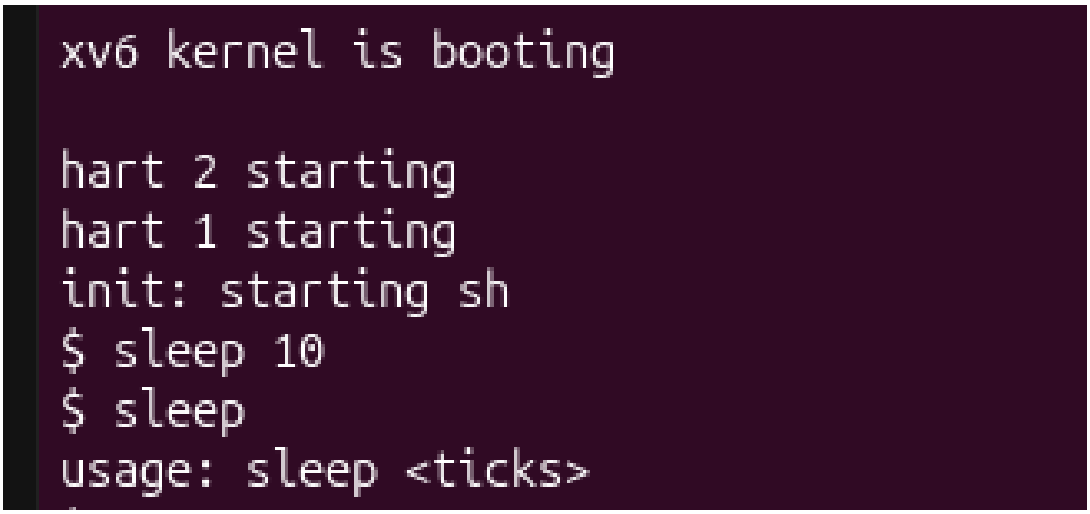


Figure 23: Results

CONCLUSION

The purpose of these two laboratory works was to develop a practical understanding of the role and internal organization of an operating system by studying it from two complementary perspectives. The first lab focused on observing Linux as a resource manager from the user's point of view, and learning how to manage files, permissions, processes, memory, devices, storage and I/O, while the second lab introduced xv6 as a small operating system, whose source code can be examined, compiled, executed, and extended. Together, these two labs helped connect the theoretical concepts of operating systems with their practical implementation.

In laboratory 1, I used WSL (Windows subsystem for Linux). We observed that Linux uses the "everything is a file" design pattern, and we saw that in action by using the same terminal interface to manage all of the aspects mentioned above. For files and permissions, we used `‘chmod 600‘` to change owner, group, and global permissions for `‘note.txt‘`. For processes, we observed how many processes existed at the moment and their statues (R for running, S for sleeping), and that PID 1 precedes all processes, since it is responsible for starting the system itself. In the memory section, we learned about swap, about the total, free, available, and resident memory. About devices and storage, we saw the size of our workspace and what the root filesystem lives on.

In laboratory 2, I used VirtualBox with OS Ubuntu 25.04. I used Linux for real this time, and we even dived deeper by using xv6, which is based on Version 6 Unix released in 1975. In this lab, we emphasized the idea of "everything is a file" design, since user programs like cat, grep, ls, kill are ordinary files. Moreover, those files don't have special privileges, they have to do system calls and let the kernel operate with the hardware. Simple features like `‘pipe‘` are built from small, general system calls combined.

Overall, from both of these labs I learned a lot of new things, such as: working with a new OS and tool, understanding a new interface design pattern, using different commands, even writing my own small program, adding and then using it to the OS.

REFERENCES

1. https://github.com/mit-pdos
2. https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf
3. https://dev.to/eteimz/everything-is-a-file-explained-g2a
4. https://tldp.org/LDP/Linux-Filesystem-Hierarchy/html/dev.html
5. https://www.interserver.net/tips/kb/understanding-swap-memory-and-when-linux-uses-it/
6. https://linuxblog.io/90-linux-commands-frequently-used-by-linux-sysadmins/